

# HSD ARENA - TEKNİK TASARIM VE GEREKSİNİM DOKÜMANI

Sürüm: 1.0

Tarih: 25 Ocak 2026

Odak: SaaS Mimarisi, Veri Bütünlüğü ve Moderasyon

## 1. YÖNETİCİ ÖZETİ VE MİMARİ VİZYON

HSD Arena; kurumların, üniversitelerin ve toplulukların kendi markalarıyla (White-Label) kullanabileceği, Subdomain tabanlı, gerçek zamanlı bir oyunlaştırma (Gamification) platformudur.

Bizi Rakiplerden Ayıran Temel Fark (USP):

Biz sadece bir yarışma aracı değiliz; kurumun dijital kimliğine bürünen bir altyapıyız. Kullanıcı banu.hsdarena.com adresine girdiğinde HSD Arena'yı değil, üniversitenin kendi geliştirdiği bir sistemi kullandığını hisseder.

**Mimari Hedefler:**

- SaaS & Multi-Tenant:** Tek veritabanı, izole edilmiş organizasyon verileri.
- Sıfır Gecikme (Zero-Latency):** 1000+ kullanıcıda bile <50ms senkronizasyon.
- Tam Denetim:** Gelişmiş moderasyon (Kick/Ban) ve hesap yaşam döngüsü yönetimi.

## 2. SİSTEM MİMARİSİ VE TEKNOLOJİ YIĞINI

Proje, geliştirici deneyimi (DX) ve yüksek performans gözetilerek aşağıdaki modern yığın üzerine inşa edilecektir.

| Katman  | Teknoloji | Seçim Nedeni (Why?)                                                                                 |
|---------|-----------|-----------------------------------------------------------------------------------------------------|
| Runtime | Bun       | Node.js yerine tercih edildi. 4x hızlı başlatma süresi ve yerleşik (native) WebSocket desteği için. |

|                     |                       |                                                                                                |
|---------------------|-----------------------|------------------------------------------------------------------------------------------------|
| <b>Backend</b>      | <b>ElysiaJS</b>       | End-to-End Type Safety sağlayan, mikro saniye bazında yanıt veren ultra hızlı API framework'ü. |
| <b>Real-Time</b>    | <b>Bun WebSockets</b> | Socket.io overhead'i olmadan, çekirdek seviyesinde optimize edilmiş bağlantı yönetimi.         |
| <b>Hot Storage</b>  | <b>Redis</b>          | Oyun durumu (State), Lobi listesi, Ban listesi ve Leaderboard gibi "uçucu" veriler için.       |
| <b>Cold Storage</b> | <b>PostgreSQL</b>     | Kalıcı veriler ve ilişkisel bütünlük (Cascade Delete) için.                                    |
| <b>ORM</b>          | <b>Drizzle</b>        | Prisma'ya göre daha hafif, SQL'e daha yakın ve sorgu performansı yüksek.                       |
| <b>Frontend</b>     | <b>React + Vite</b>   | Bileşen bazlı mimari ve hızlı derleme.                                                         |
| <b>Styling</b>      | <b>Tailwind CSS</b>   | CSS Değişkenleri (var(--primary)) kullanılarak dinamik tema (White-Label) yönetimi.            |
| <b>DevOps</b>       | <b>Docker + Nginx</b> | Konteynerizasyon ve Wildcard Subdomain (*.hsdarena.com) yönlendirmesi.                         |

---

### 3. FONKSİYONEL GEREKSİNİMLER

Bu bölüm, HSD Arena platformunun çok katmanlı yetki mimarisini, **çoklu organizasyon desteğini**, hesap yaşam döngüsünü ve oyun motoru yeteneklerini tanımlar.

#### A. KİMLİK, KAYIT VE ORGANİZASYON YÖNETİMİ

Sistem, kullanıcıların ve organizasyonların tamamen ayrıştırıldığı, "**Tek Hesap, Çok Organizasyon**" (**One User, Many Orgs**) prensibiyle çalışan gerçek bir Multi-Tenant yapısındadır.

##### 1. Merkezi Kimlik ve Çoklu Üyelik (Global Passport)

- **Kayıt:** Tüm kullanıcılar hsdarena.com üzerinden tek bir kez hesap oluşturur. Bu hesap kullanıcının "Pasaport"udur.
- **Organizasyona Dahil Olma (Davet):** Organizasyon Super Admini, kullanıcı adını aratarak kişiyi davet eder.
  - *Kritik:* Bir kullanıcı **aynı anda** birden fazla organizasyona üye olabilir. (Örn: Hem "HSD Topluluğu"nda Yönetici, hem "Kendi Şirketi"nde Super Admin olabilir).
- **Giriş ve Organizasyon Seçimi (Context Switching):**
  - Kullanıcı giriş yaptığında, üyesi olduğu organizasyonların listesini görür.
  - Çalışmak istediği organizasyonu seçer ve sistem o organizasyondaki yetkilerini yükler.
  - Panel içerisinde organizasyonlar arası geçiş yapılabilir.

##### 2. Organizasyon Kurulumu

- **Süreç:** Herhangi bir kullanıcı (başka bir yerde yönetici olsa bile), ana dashboard üzerinden "Yeni Organizasyon Oluştur" diyebilir.
- **Kapsam:** Oluşturan kişi, o organizasyonun **Super Admin**'i olur. Bu durum diğer organizasyonlardaki rollerini etkilemez.

##### 3. Hesap ve Profil Yönetimi (CRUD)

- **Profil Düzenleme:** Kullanıcı adı, şifre ve email değişiklikleri globaldir ve tüm organizasyonlardaki kimliğini günceller.
- **Hesap Silme (Bireysel):** Kullanıcı hesabını sildiğinde, üye olduğu tüm organizasyonlardan kaydı düşer (Members tablosundan silinir).

## B. ROL VE YETKİ HİYERARŞİSİ (RBAC - BAĞLAMSAK)

Yetkiler **evrensel deęil, organizasyon bazlıdır**. Kullanıcının yetkisi, o an aktif olan organizasyon bağlamına (Context) göre belirlenir ve organizasyon içinde Sistemde yetkiler yukarıdan aşağıya (Super Admin > Admin > Yönetici) kapsayıcı bir şekilde ilerler..

### 1. Organizasyon Sahibi (Super Admin)

- **Tanım:** O organizasyonun kurucusu ve sahibidir.
- **Benzersiz Yetkiler:**
  - Organizasyonu silebilir (Dięer organizasyonlardaki üyelikleri etkilenmez).
  - Kullanıcı davet eder, yetki verir/alır veya organizasyondan çıkarır.
  - Kurumsal özelleştirme (White-Label) yapar.
- **Kritik İşlem: Zincirleme Silme (Cascade Deletion):**
  - Eğer Super Admin hesabını tamamen silerse; **sahibi olduęu Organizasyonlar** silinir.
  - Ancak **üyesi olduęu dięer organizasyonlar** silinmez, sadece o organizasyonlardaki üyelięi sonlanır.
- **Kapsayıcı Yetkiler:** Quiz oluşturma, oyun yönetme ve katılımcı atma dahil tüm alt yetkilere sahiptir.

### 2. Organizasyon Admini (Yetkilendirilmiş Yönetici)

- **Tanım:** Super Admin tarafından o organizasyon özelinde yetkilendirilmiş kişidir.
- **Ekstra Yetkiler:**
  - **Kurumsal Özelleştirme (White-Label):** Organizasyonun logosunu, arka planını ve renklerini deęiştirebilir.
- **Kısıtlamalar:** Organizasyonu silemez organizasyonana yönetici davet edemez. Organizasyondan ayrılırsa sadece o kurumdaki yetkisi biter.
- **Kapsayıcı Yetkiler:** Quiz oluşturma, oyun yönetme ve katılımcı atma dahil tüm alt yetkilere sahiptir.

### 3. Yönetici (Manager)

- **Tanım:** O organizasyondaki operasyonel kullanıcıdır.
- **Operasyonel Yetkiler:**
  - **Quiz Yönetimi:** Quiz oluşturma, düzenleme ve ayarlarını yapma.
  - **Soru Yönetimi:** Soru ekleme/silme/düzenleme/sorunun doğru sikini secme vesoruların yerlerini deęistirme.
  - **Moderasyon Yetkisi:** Kendi başlattıęı oyunlarda katılımcı atabilir (Kick).

## C. QUIZ VE OYUN MOTORU GEREKSİNİMLERİ

Oyun deneyimi, Quiz oluşturulurken belirlenen ayarlara göre şekillenir.

### 1. Quiz Ayarları ve Mod Seçimi (Persistent Configuration)

Oyun modu (Sahne/Kişisel) her oyun başlatıldığında sorulmaz; Quiz'in kalıcı bir ayarıdır.

- **Oluşturma Anı:** Yönetici yeni quiz oluştururken **Varsayılan Oyun Modu**'nu (STAGE veya PERSONAL) seçmek zorundadır.
- **Quiz Ayarları Menüsü:** Quiz kartındaki ayarlar bölümünden bu mod daha sonra değiştirilebilir.
  - **Kişisel Mod (Personal):** Sorular ve şıklar katılımcının telefonunda görünür (Uzaktan katılım).
  - **Sahne Modu (Stage):** Telefonda sadece butonlar görünür, soru dev ekrandadır (Fiziksel etkinlik).

### 2. Oyun Başlatma ve Akış

- **Başlatma:** Yönetici "Başlat" butonuna bastığında, sistem soru sormaz; veritabanındaki kayıtlı modu (default\_mode) okur ve oyunu o modda başlatır.
- **Lobi ve QR Kod:** Oyun başladığında benzersiz PIN ve otomatik QR Kod üretilir.
- **Moderasyon (Kick/Ban):** Yetkililer lobi veya oyun esnasında kullanıcı atabilir. Atılan kullanıcının bağlantısı kesilir ve IP adresi o oturum için kara listeye alınır.
- **İlerleme:** Oyun otomatik akmaz; "Sonraki Soru" kontrolü yöneticidedir. Her sorunun süresinin bitmesinden ya da süre bitmeden kullanıcıların hepsinin soruları cevaplamasından sonra skorbord ekranı döner.

### 3. Gerçek Zamanlı Senkronizasyon

- Yetkili "Başlat" butonuna bastığı anda (WebSocket QUESTION\_START), kayıtlı moda göre 1000+ katılımcının ekranı eş zamanlı (Zero-Latency) güncellenir.

## D. KATILIMCI (PLAYER) ARAYÜZÜ

### 1. Giriş ve Kimlik

- **Manuel Giriş:** Katılımcılar PIN Kodu ve Nickname girerek dahil olur.
- **Hızlı Giriş (QR Scan):** Lobi ekranındaki QR Kodu okutarak PIN girmeden doğrudan nick belirleme ekranına geçerler.
- **Validasyon:** Aynı isim varsa sistem sonuna rakam ekler (Yunus#1). Atılan kullanıcılar tekrar giremez.

### 2. Puanlama Sistemi

- **Puanlama:** Puan = Baz Puan + (Kalan Süre x Çarpan).
- **Streak (Alev):** Üst üste 3 doğru cevap veren katılımcı "Alev Modu"na girer ve x1.5 kat puan kazanır.

## 4. VERİ TABANI ŞEMASI VE İLİŞKİLER

Bu mimari, "**Members**" (Üyeler) ara tablosu sayesinde kullanıcıların ve organizasyonların bağımsızlığını korur, ancak yetkilerin organizasyon bazlı (Scoped) yönetilmesini sağlar.

### A. PostgreSQL Şeması (Kalıcı Veri - Cold Storage)

Veri bütünlüğü, ON DELETE CASCADE ve UNIQUE kısıtlamaları (constraints) ile veritabanı seviyesinde zorunlu kılınmıştır.

#### 1. Tablo: organizations (Kurumlar)

Bağımsız varlıklardır.

| Sütun Adı | Veri Tipi | Kısıtlamalar (Constraints) | Açıklama                                                                      |
|-----------|-----------|----------------------------|-------------------------------------------------------------------------------|
| id        | uuid      | PK, defaultRandom          | Benzersiz organizasyon kimliği.                                               |
| name      | varchar   | NOT NULL                   | Kurum adı (örn: HSD Topluluğu).                                               |
| subdomain | varchar   | UNIQUE, INDEX              | <b>Kritik:</b> banu, google gibi alan adları. Routing hızı için indexlenmeli. |
| branding  | jsonb     | DEFAULT '{}'               | { "logo": "url", "colors": { "primary": "#Hex" } }. White-Label ayarları.     |

|            |           |               |                                                                                                                                                 |
|------------|-----------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| owner_id   | uuid      | FK(users.id)  | <b>ON DELETE CASCADE.</b><br><br><i>Mantık:</i><br>Organizasyon sahibi (Super Admin) hesabını silerse, bu organizasyon ve tüm verileri silinir. |
| created_at | timestamp | DEFAULT now() | Oluşturulma tarihi.                                                                                                                             |

## 2. Tablo: users (Kullanıcılar - Global Pasaport)

Sadece kimlik bilgilerini tutar. Rol veya organizasyon bilgisi içermez.

| Sütun Adı  | Veri Tipi | Kısıtlamalar      | Açıklama                                   |
|------------|-----------|-------------------|--------------------------------------------|
| id         | uuid      | PK, defaultRandom | Benzersiz kullanıcı kimliği (Passport ID). |
| username   | varchar   | UNIQUE, NOT NULL  | Davet ve giriş için kullanılır.            |
| email      | varchar   | UNIQUE, NOT NULL  | İletişim adresi.                           |
| password   | varchar   | NOT NULL          | Argon2 ile hashlenmiş şifre.               |
| created_at | timestamp | DEFAULT now()     | Kayıt tarihi.                              |

### 3. Tablo: members (Organizasyon Üyeleri - Yetki Tablosu)

Kullanıcıyı organizasyona bağlayan ve rolünü tanımlayan \*\*\*Link Table\*\*\*dır.

| Sütun Adı         | Veri Tipi | Kısıtlamalar                            | Açıklama                                                                                           |
|-------------------|-----------|-----------------------------------------|----------------------------------------------------------------------------------------------------|
| id                | uuid      | PK, defaultRandom                       | Benzersiz üyelik kaydı.                                                                            |
| org_id            | uuid      | FK(organizations.id )                   | <b>ON DELETE CASCADE.</b><br><br><i>Mantık:</i><br>Organizasyon silinirse, üye listesi temizlenir. |
| user_id           | uuid      | FK(users.id)                            | <b>ON DELETE CASCADE.</b><br><br><i>Mantık:</i> Kullanıcı hesabını silerse, üyelikleri silinir.    |
| role              | enum      | 'SUPER_ADMIN',<br>'ADMIN',<br>'MANAGER' | <b>Kritik:</b> Kullanıcının <i>bu organizasyondaki</i> yetkisi.                                    |
| joined_at         | timestamp | DEFAULT now()                           | Katılım tarihi.                                                                                    |
| <b>Constraint</b> | -         | UNIQUE(org_id, user_id)                 | Bir kullanıcı aynı organizasyona iki kere eklenemez.                                               |

**4. Tablo: quizzes (Yarışmalar)**  
İçerik tablosudur.

| Sütun Adı    | Veri Tipi | Kısıtlamalar                  | Açıklama                                                                                                      |
|--------------|-----------|-------------------------------|---------------------------------------------------------------------------------------------------------------|
| id           | uuid      | PK, defaultRandom             | Benzersiz quiz kimliği.                                                                                       |
| org_id       | uuid      | FK(organizations.id )         | <b>ON DELETE CASCADE.</b><br><br><i>Mantık:</i> Organizasyon silinirse quizler de silinir.                    |
| creator_id   | uuid      | FK(users.id)                  | <b>ON DELETE SET NULL.</b><br><br><i>Mantık:</i> Kullanıcı silinse bile quiz kalır, "Oluşturan: Anonim" olur. |
| title        | varchar   | NOT NULL                      | Quiz başlığı.                                                                                                 |
| default_mode | enum      | 'PERSONAL', 'STAGE', NOT NULL | <b>Kalıcı Ayar:</b> Quiz oluşturulurken seçilen varsayılan oyun modu.                                         |
| is_deleted   | boolean   | DEFAULT false                 | Soft Delete (Çöp kutusu mantığı) için.                                                                        |

##### 5. Tablo: questions (Sorular)

Quize sıkı sıkıya bağlıdır.

| Sütun Adı     | Veri Tipi | Kısıtlamalar      | Açıklama                                                                                       |
|---------------|-----------|-------------------|------------------------------------------------------------------------------------------------|
| id            | uuid      | PK, defaultRandom | Benzersiz soru kimliği.                                                                        |
| quiz_id       | uuid      | FK(quizzes.id)    | ON DELETE CASCADE. Quiz silinirse sorular gider.                                               |
| text          | text      | NOT NULL          | Soru metni.                                                                                    |
| media_url     | text      | NULLABLE          | Görsel varsa URL'i.                                                                            |
| time_limit    | integer   | DEFAULT 30        | Saniye cinsinden süre.                                                                         |
| points        | integer   | DEFAULT 1000      | Baz puan değeri.                                                                               |
| order_index   | integer   | NOT NULL          | Sıralama numarası.                                                                             |
| options       | jsonb     | NOT NULL          | <b>Şık Listesi:</b><br>["Ankara", "İstanbul", "İzmir", "Bursa"] şeklinde dizi tutar.           |
| correct_index | integer   | NOT NULL          | <b>Doğru Cevap:</b><br>Dizideki doğru şıkkın sırasıdır (Index). (Örn: 0 ise ilk şık doğrudur). |

## 6. Tablo: invitations

Kullanıcı ile organizasyon arasında kurulan, onaylandığında kalıcı üyeliğe dönüşecek olan 'Geçici Köprü (Pending Link)' tablosudur

| Sütun Adı  | Veri Tipi | Kısıtlamalar               | Açıklama                                                  |
|------------|-----------|----------------------------|-----------------------------------------------------------|
| id         | uuid      | PK, defaultRandom          | Benzersiz davetiye ID'si.                                 |
| org_id     | uuid      | FK(organizations.id)       | Hangi kuruma çağırılıyor? (ON DELETE CASCADE)             |
| inviter_id | uuid      | FK(users.id)               | Daveti atan yönetici (ON DELETE CASCADE)                  |
| invitee_id | uuid      | FK(users.id), NOT NULL     | Davet edilen kişi.                                        |
| status     | enum      | DEFAULT 'PENDING'          | 'PENDING', 'ACCEPTED', 'REJECTED'.                        |
| created_at | timestamp | DEFAULT now()              | Oluşturulma tarihi (Süre hesabı bunun üzerinden yapılır). |
| -          | -         | UNIQUE(org_id, invitee_id) | Bir kişiye aynı yerden tek davet.                         |

## B. Redis Şeması (Oyun Belleği & Durum - Hot Storage)

Oyun anında veritabanı (PostgreSQL) yorulmaz, her şey Redis üzerinde döner. Doğrulama (Validation) işlemi, Redis'e yüklenen "Cevap Anahtarı" üzerinden yapılır.

**Anahtar Formatı:** game:{pin}:{type}

### 1. Oyun Durumu (Game State)

Oyunun o anki konfigürasyonunu ve akışını tutar.

- **Key:** HASH: game:{pin}:state
- **Fields:**
  - status: LOBBY | ACTIVE | SCOREBOARD | FINISHED
  - currentQuestionIndex: 0, 1... (Şu an kaçınıcı sorudayız?)
  - mode: PERSONAL | STAGE (Yönetici seçiminden gelir).
  - hostSocketId: Yöneticinin socket ID'si.
  - totalAnswers: O anki soruya o ana kadar kaç kişi cevap verdi? (Otomatik ilerleme için).

### 2. Oyuncu Listesi (Players)

Lobideki ve oyundaki aktif oyuncuların listesi.

- **Key:** SET: game:{pin}:players
- **Values:** [ "socket\_id\_1", "socket\_id\_2" ]
- **Amaç:** QUESTION\_START anında herkese soru verisini (şıklarıyla beraber) göndermek için.

### 3. Oyuncu Detayları (Player Info)

Her oyuncunun anlık performans verisi.

- **Key:** HASH: game:{pin}:player:{socketId}
- **Fields:**
  - nickname: "Yunus#1"
  - score: 1450 (Toplam Puan)
  - streak: 3 (Alev modu için kontrol edilir)
  - ip: Güvenlik ve loglama.

### 4. Cevap Anahtarı (Answer Key) — [YENİ EKLENDİ]

Sunucunun elindeki "Kopya Kağıdı"dır. Oyun başladığında Postgres'ten çekilen **doğru şık indeksleri** buraya yazılır.

- **Key:** HASH: game:{pin}:answers
- **Field:** {question\_id} (Sorunun UUID'si)
- **Value:** {correct\_index} (Doğru şıkkın sırası - Integer)
  - *Örnek:* "soru-uuid-123": "0" (Demek ki bu sorunun doğru cevabı ilk şık).
- **İşleyiş:**
  - Kullanıcı SUBMIT\_ANSWER olayı ile { questionId: "...", answerIndex: 0 } yollar.
  - Backend, Redis'teki bu Hash'ten questionId'ye karşılık gelen doğru indexi çeker.
  - Gelen Index == Redis Index ise puan verir.

## 5. Canlı Skor Tablosu (Leaderboard)

Sıralamayı tutar.

- **Key:** ZSET: game:{pin}:leaderboard
- **Score:** Total Points (Puan)
- **Member:** Nickname (Kullanıcı Adı)
- **Komut:** ZREVRANGE 0 4 ile anlık ilk 5 çekilir.

## 6. Yasaklı Listesi (Moderation)

- **Key:** SET: game:{pin}:banned
- **Values:** [ "IP\_Address" ]



## Nasıl Çalışır? (Özet Senaryo)

- **Yükleme (Oyun Başlarken):**
    - Yönetici "Başlat" der.
    - Backend, PostgreSQL'deki questions tablosundan o quizin sorularını çeker.
    - Her sorunun id ve correct\_index bilgisini alıp Redis'teki game:{pin}:answers Hash'ine yazar.
  - **Oynama (Yarışmacı):**
    - Yarışmacının telefonunda 4 şık (0, 1, 2, 3) görünür.
    - Yarışmacı "A" şıkkına (Index: 0) basar.
  - **Doğrulama (Redis):**
    - Backend hemen Redis'e sorar: HGET game:{pin}:answers {soru\_id}
    - Redis cevap verir: "0"
    - Backend bakar: Kullanıcı 0 yolladı, Doğrusu 0.
    - **SONUÇ:** Doğru! Puan hesaplanır, leaderboard güncellenir.
-

## 5. KRİTİK İŞ AKIŞLARI

Bu bölüm, veri bütünlüğünü korumak ve sistem güvenliğini sağlamak için arka planda çalışan karmaşık senaryoları tanımlar.

### Senaryo 1: Super Admin (Owner) Hesabını Siliyor [Zincirleme Silme]

Bu işlem, sistemdeki en yıkıcı veri operasyonudur. "Tek Hesap, Çok Organizasyon" yapısında şu şekilde işlenir:

- **Tetikleyici:** Organizasyon Sahibi (Super Admin) panelden "*Hesabımı Sil*" butonuna basar ve şifresini girerek onaylar.
- **Veritabanı Tetikleyicisi (Cascade Chain):**
  - Backend, DELETE FROM users WHERE id = {OwnerID} komutunu çalıştırır.
  - **Adım 1 (Organizasyonun Silinmesi):** organizations tablosundaki owner\_id kısıtlaması (ON DELETE CASCADE) devreye girer ve **Organizasyon silinir.**
  - **Adım 2 (Üyeliklerin Düşmesi):** Organizasyon silindiği için, members tablosunda o organizasyona ait (WHERE org\_id = {OrgID}) **tüm üyelik kayıtları otomatik silinir.**
    - **Sonuç:** O kurumda çalışan Admin ve Yöneticilerin hesapları (users tablosu) SİLİNMEZ. Sadece o organizasyonla olan bağları kopar ve yetkileri düşer.
  - **Adım 3 (İçeriğin Silinmesi):** Organizasyon silindiği için, quizzes tablosundaki (ON DELETE CASCADE) kuralı çalışır ve **tüm quizler/sorular silinir.**
- **Nihai Durum:** Organizasyon ve verileri yok olur. Eski çalışanlar "Bağımsız Kullanıcı" statüsüne döner. Sistemde yetim (orphan) veri kalmaz.

### Senaryo 2: Moderasyon (Kullanıcı Atma/Banlama)

Oyun esnasında düzeni bozan katılımcıları engellemek için Socket ve Redis entegre çalışır.

- **Tetikleyici:** Yönetici (Manager), lobi veya oyun ekranında katılımcı isminin yanındaki "**X**" (**At**) butonuna basar.
- **Backend İşlemi:**
  - Sunucu, atılacak oyuncunun **Socket ID'sini** ve **IP Adresini** tespit eder.
  - **Adım 1 (Bağlantı Kesme):** Hedef sokete FORCE\_DISCONNECT mesajı yollanır ve sunucu taraflı bağlantı (socket.terminate()) zorla koparılır.
  - **Adım 2 (Kara Liste):** Oyuncunun IP adresi veya Cihaz İzi, Redis üzerindeki SET: game:{pin}:banned listesine eklenir.
- **Frontend ve Tekrar Giriş:**
  - Atılan oyuncunun ekranında: "*Yönetici tarafından oyundan uzaklaştırıldınız.*" uyarısı

belirir.

- Oyuncu QR kodu tekrar okutup girmeye çalışırsa; JOIN\_ROOM servisi Redis Ban listesini kontrol eder ve **"Yasaklısınız"** diyerek içeri almaz.

### Senaryo 3: Organizasyon Değişirme (Context Switching)

Kullanıcının panelde bir kurumdan diğerine geçiş yapmasıdır.

- **Tetikleyici:** Kullanıcı üst menüden *"HSD Topluluğu"* yerine *"Kendi Şirketim"* seçeneğine tıklar.
- **Backend İşlemi:**
  - Frontend, yeni org\_id ile /auth/switch-org (veya yeni token isteği) yapar.
  - Backend, members tablosundan kullanıcının o organizasyondaki **Rolünü** (role) çeker.
  - Yeni bir **JWT Token** (veya Session Scope) oluşturulur. Bu token sadece seçilen organizasyon için geçerlidir.
- **Sonuç:** Kullanıcının ekranı yenilenir; menüler ve yetkiler yeni organizasyona (Örn: Yönetici -> Super Admin) göre güncellenir.

## 6. API VE WEBSOCKET MİMARİSİ

HSD Arena, yönetimsel işlemler için **RESTful API**, gerçek zamanlı oyun senkronizasyonu için **Native WebSocket** protokolü kullanır. Aşağıdaki endpoint tanımları, Swagger dokümantasyonundan ve sistem mimarisinden türetilmiştir.

### 6.1. API Genel Standartları

Tüm HTTP yanıtları standart bir zarf (envelope) yapısına sahiptir.

- **Base URL:** http://localhost:3000/api/v1 (Dev)
- **Authentication:** Authorization: Bearer <JWT\_TOKEN> header'ı zorunludur (Login hariç).
- **Response Format:**

```
JSON
{
  "success": true,    // İşlem durumu (Boolean)
  "data": { ... },    // Dönen veri objesi (Null olabilir)
  "message": "string" // Bilgilendirme mesajı
}
```

## 6.2. KİMLİK VE KULLANICI YÖNETİMİ (AUTH & USERS)

Kullanıcıların sisteme kaydı ve girişi "Global Passport" mantığıyla yönetilir.

### A. Auth Operations

- **POST /login**
  - **Amaç:** Kullanıcıyı doğrular ve JWT token üretir.
  - **Body (JSON):**
    - email (string, email, **required**)
    - password (string, min:6, **required**)
  - **Response:**  
JSON

```
{
  "success": true,
  "data": {
    "token": "jwt_string...",
    "user": { "id": "uuid", "username": "...", "email": "..." }
  }
}
```

### B. User Operations

- **POST /users (Register)**
    - **Amaç:** Yeni kullanıcı oluşturur ve otomatik token döner.
    - **Body:**
      - username (string, min:3, **required**)
      - email (string, email, **required**)
      - password (string, min:6, **required**)
  - **GET /users**
    - **Amaç:** Sistemdeki tüm kullanıcıları listeler.
    - **Response:** data: { "users": [Array] }
  - **GET /users/{id}**
    - **Amaç:** ID'si verilen tekil kullanıcının detaylarını getirir.
  - **PATCH /users/{id}**
    - **Amaç:** Kullanıcı bilgilerini günceller.
    - **Body:** name, email, password (Hepsi opsiyonel gönderilebilir).
  - **DELETE /users/{id}**
    - **Amaç:** Kullanıcıyı sistemden siler.
    - **Header:** Authorization zorunludur.
-

## 6.3. ORGANİZASYON YÖNETİMİ (ORGANIZATIONS)

Sistem **Subdomain** tabanlı çalıştığı için path parametrelerinde genellikle id yerine orgDomain kullanılır.

- **POST /org**
  - **Amaç:** Yeni bir organizasyon (Tenant) oluşturur.
  - **Body:**
    - name (string, **required**)
    - subdomain (string, min:3, **required**)
    - branding: { "logoUrl": "string" } (object, optional)
- **GET /org/{orgDomain}**
  - **Amaç:** Subdomain (örn: hsd) üzerinden kurumun public bilgilerini çeker.
- **GET /org**
  - **Amaç:** Kullanıcının görebildiği tüm organizasyonları listeler.
- **PATCH /org/{orgDomain}**
  - **Amaç:** Kurum bilgilerini ve markalamayı günceller.
- **DELETE /org/{orgDomain}**
  - **Amaç:** Kurumu ve bağlı tüm verileri (Cascade) siler.

---

## 6.4. DAVETİYE SİSTEMİ (INVITATIONS)

Kullanıcıları kurumlara bağlayan köprüdür.

- **POST /org/{orgDomain}/invitations**
    - **Amaç:** Bir kullanıcıyı kuruma davet eder.
    - **Body:**
      - inviteeUsername (string, **required**)
    - **Backend Mantığı:** inviterId (davet eden) cookie/token'dan alınır. inviteeId, gönderilen username üzerinden veritabanında bulunur.
  - **GET /org/{orgDomain}/invitations**
    - **Amaç:** O kurumdaki bekleyen/onaylanan tüm davetiyeleri listeler.
  - **PATCH /org/{orgDomain}/invitations/{invitationId}**
    - **Amaç:** Davet durumunu günceller (Kabul/Red).
    - **Body:** status (string -> 'ACCEPTED' | 'REJECTED').
  - **DELETE /org/{orgDomain}/invitations/{invitationId}**
    - **Amaç:** Gönderilmiş bir daveti iptal eder veya siler.
-

## 6.5. İÇERİK YÖNETİMİ (QUIZZES & QUESTIONS)

Oyun içeriğinin oluşturulduğu CMS katmanıdır.

### A. Quiz Operations

Path: /org/{orgDomain}/quizzes/

- **POST** /.../quizzes
  - **Amaç:** Yeni yarışma oluşturur.
  - **Body:**
    - title (string, **required**)
    - defaultMode (enum: PERSONAL | STAGE, **required**) -> Oyunun varsayılan modu burada belirlenir.
- **GET** /.../quizzes
  - **Amaç:** Kuruma ait tüm quizleri listeler.
- **GET** /.../quizzes/{quizId}
  - **Amaç:** Tekil quiz detayını getirir.
- **PATCH** /.../quizzes/{quizId}
  - **Amaç:** Başlık veya oyun modunu günceller.
- **DELETE** /.../quizzes/{quizId}
  - **Amaç:** Quizi siler (Soft veya Hard delete).

### B. Question Operations

Path: /org/{orgDomain}/quizzes/{quizId}/questions/

1. **POST** /.../questions
    - **Amaç:** Quize soru ekler. **Body verisi çok kritiktir.**
    - **Body:**
      - text (string, **required**)
      - timeLimit (number, default: 30, **required**)
      - points (number, default: 1000)
      - options (array string[], **required**) -> Ör: ["A şıkkı", "B şıkkı"]
      - correctIndex (number, **required**) -> Doğru şıkkın dizideki indexi (0, 1...).
      - orderIndex (number, **required**) -> Sorunun sırası.
      - mediaUrl (string, optional)
  2. **PATCH** /.../questions/{questionId}
    - **Amaç:** Soru metnini veya şıkları günceller.
-

## 6.6. OYUN MOTORU (GAMES ENGINE - HTTP)

Oyunun Redis üzerindeki oturumunu yöneten "Setup" katmanıdır.

1. **POST** /games
  - **Yetki:** Sadece Yönetici.
  - **Amaç:** Canlı oyun oturumunu başlatır.
  - **Body:** { "quizId": "uuid" }
  - **İşlem:**
    - Quiz'in defaultMode (STAGE/PERSONAL) ayarını çeker.
    - 6 haneli benzersiz **PIN** üretir.
    - Redis'te game:{pin}:state anahtarını oluşturur (LOBBY modunda).
  - **Response:** { "success": true, "gameId": "...", "pin": "482910", "mode": "STAGE" }
2. **DELETE** /games/{id}
  - **Amaç:** Oyunu bitirir ve Redis verisini temizler.
3. **GET** /games/{id}
  - **Amaç:** Oyun bittikten sonra rapor ekranı için özet bilgileri (Kazanan, Toplam Oyuncu) döner.

## 6.7. OYUN MOTORU (GAMES ENGINE - WEBSOCKET)

**Endpoint:** ws://api.hsdarena.com/games/{id}/ws

**Mimari:** Event-Driven (Olay Güdümlü).

### A. Client -> Server (İstekler)

| Event         | Payload           | Açıklama                                                                     |
|---------------|-------------------|------------------------------------------------------------------------------|
| JOIN_ROOM     | { pin, nickname } | Oyuncu girmek ister. Ban kontrolü yapılır. İsim çakışırsa sonuna #1 eklenir. |
| START_GAME    | { gameId }        | <b>Sadece Host.</b> Oyunu ACTIVE moda çeker ve ilk soruyu yayınlar.          |
| SUBMIT_ANSWER | { qId, ansIdx }   | Oyuncu cevap verir. Puan hesaplanır: Baz + (Süre * Çarpan).                  |

|               |            |                                                                       |
|---------------|------------|-----------------------------------------------------------------------|
| NEXT_QUESTION | { gameld } | <b>Sadece Host.</b> Manuel olarak bir sonraki soruya geçişi tetikler. |
|---------------|------------|-----------------------------------------------------------------------|

## B. Server -> Client (Yayınlar)

| Event           | Hedef       | Payload                              | Açıklama                                                                                                |
|-----------------|-------------|--------------------------------------|---------------------------------------------------------------------------------------------------------|
| JOIN_SUCCESS    | Tekil       | { status: "WAITING", myNick: "..." } | Giriş başarılı onayı.                                                                                   |
| LOBBY_UPDATE    | <b>Host</b> | { count: 45 }                        | <b>Kritik:</b> Sadece Host ekranındaki sayacı günceller. Oyunculara bildirim gitmez (Sessiz Lobi).      |
| QUESTION_START  | Herkes      | { text, options[], time }            | Soruyu ekrana basar. <b>Mod Farkı:</b> STAGE modunda options metinleri boş gider, sadece renkler gider. |
| SHOW_SCOREBOARD | Herkes      | { top5: [...] }                      | Süre bittiğinde veya herkes cevapladığında otomatik tetiklenir.                                         |
| GAME_OVER       | Herkes      | { winner: "..." }                    | Oyun bitiş ekranı.                                                                                      |